VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT 1

Infinity Castle Battle

INTRODUCTION TO COMPUTING
Class: CC05

STUDENT

Full Name	Student ID
Hắc Minh Quân	2550201

November, 2025

TABLE OF CONTENTS

1. Introduction **3**
1.1. Preface 3
1.2. Overview 4
2. Theory **4**
3. Methodology **5**
3.1. Glossary 5
3.2. Algorithm 5
3.3. Code Structuring 6
3.4. Testing 6
4. Implementation and Explanation **6**
4.1. Header 8
4.2. Introduction Output 8
4.3. Opening the YouTube Theme 8
4.4. Variable Declaration and Data Input 9
4.5. Scene 1: Power Classification 10
4.6. Scene 2: Infinity Castle Gate 10
4.7. Scene 3: Battle Strategy 11
4.8. Scene 4: Final Outcome 12
5. Results 13
5.1. Example 1 13
5.2. Example 2 13
5.3. Example 3 14
5.4. Example 4 14
5.5. Example 5 15
6. Conclusion **16**
6.1. Finale 16
6.2. Epiloque 17
7. AI-used Declaration **17**

1. Introduction

1.1. Preface

Darkness breathes within the walls of infinity…

The camera drifts through a void of floating corridors - endless halls twisting like a labyrinth in the sky. Lanterns flicker crimson. The floor ripples as though alive. From every shadow, a whisper echoes - the sound of demons stirring after centuries of silence.

The **Infinity Castle** is not merely a place; it is a will. It shifts, adapts, and hungers. Each path leads deeper into despair, each door opens toward a new trial.

Then - a faint light.
Footsteps echo.
A lone **Demon Slayer** stands at the threshold. His blade gleams with cold resolve. The mark of countless battles lines his face - scars written by survival itself.

He breathes in.

One breath… of calm before chaos.

In his hand, the Talisman hums faintly - the last key that keeps the Castle's curse from consuming him. Around him, the walls bend, whispering names of those who entered before and never returned.

Somewhere in the depths, the Demon Lord waits - power unmatched, eyes glimmering with hunger for a final fight.

But the Slayer does not flinch.
He has prepared. His strength, skill, and spirit are all that stand between dawn and eternal night.

The air trembles.
A deep drumbeat resounds - thoom… thoom… thoom… - like the Castle's own heartbeat.

The Battle begin!

Now, choices matter.
Every breath, every strike, every decision determines fate.
Will he rise as a Hashira, face the darkness head-on, or fall and vanish into the Castle's endless void?

The gates creak open -
light fades -
and the Infinity Castle awaits its challenger…

1.2. Overview

This project investigates the design, development, and execution of a C++ program titled **The Infinity Castle Battle** in Demon Slayer universe.

The program integrates arithmetic computation, conditional logic (if-else), and multi-scene decision structures to simulate a sequential battle scenario. It also demonstrates cross-platform capability and interactive system control by opening an external soundtrack link.

By employing well-organized code structure, clear user interaction, and precise logic modeling, this program fulfills the educational objectives in algorithmic design, computation, and structured programming.

2. Theory

The theoretical basis for this project lies in the following core programming principles:

- **Procedural Programming**: a top-down logic structure, beginning with variable initialization, input collection, and progressing through multiple conditional evaluations.
- **Input and Output Management**: along with formatting manipulators like `setprecision(1)`
- **Conditional branching**: Multiple if-else and ternary (`?:`) operators manage decision-making.
- **Cross-platform Integration**: Preprocessor directives (`#ifdef _WIN32, #elif __APPLE__`) allows the code to execute platform-specific commands (opening URLs).
- **Thread synchronization**: The inclusion of `std::this_thread::sleep_for()` demonstrates controlled program timing, improving user experience with a cinematic pause.
- **Arithmetic modeling**: Mathematical expressions determine attributes: power level, advantage, and battle outcomes, representing algorithmic thinking applied to narrative logic.

3. Methodology

The program was implemented using the **C++17**

3.1. Glossary

Variable	Data Type	Meaning
`slayerLevel`	`int`	Demon slayer's level (1-10)
`hp`	`int`	Current health points of the demon slayer
`breathingMastery`	`double`	Breathing Technique mastery level (0.0-1.0)
`hasTalisman`	`int`	Indicates if the slayer has a talisman to open the gate (`0`: no, `1`: yes)
`timeOfDay`	`char`	Time of day, `D` = daytime, `N` = night.
`demonPresence`	`int`	Indicates if a demon is present at the gate (`0`: no, `1`: yes)
`demonRank`	`int`	Rank of the Upper Moon (1-6)
`swordSharpness`	`double`	Sharpness of the Nichirin sword (0-100)
`allyCount`	`int`	Number of allies assisting in battle
`bossHP`	`int`	Initial HP of the boss in Infinity Castle
`totalDamage`	`int`	Total damage dealt by the slayer during the fight
`specialMoveReady`	`int`	Indicates if the ultimate move is ready (`0`: not ready, `1`: ready)

Table 1: List of variables and their meanings in the *Infinity Castle* program

3.2. Algorithm

The simulation was divided into four logical segments (Scenes 1-4), each with a clearly defined computational goal:

- Scene 1: Rank classification through computed power index.
- Scene 2: Entry decision based on contextual inputs (time, talisman, demon presence).
- Scene 3: Strategy determination based on advantage calculation.
- Scene 4: Final outcome evaluation after total damage analysis.

3.3. Code Structuring

Each scene is implemented sequentially, with properly declared variables, explicit type management, and formatted console output for readability.

3.4. Testing

The program was tested using 5-sample input sets to ensure accuracy in rank classification, conditional branching, and numerical computation.

4. Implementation and Explanation

```cpp
#include <iostream>
#include <iomanip>
#include <string>
#include <thread>
#include <chrono>
#ifdef _WIN32
#include <windows.h>
#endif

using namespace std;

int main() {

    cout << fixed << setprecision(1);

    cout << "\nWelcome come to THE INFINITY CASTLE !!\n\n";
    cout << "To survive, please enter (๑•`ㅂ•´)و✧ \n";
    cout << "slayerLevel hp breathingMastery hasTalisman timeOfDay demonPresence ";
    cout << "demonRank swordSharpness allyCount bossHP totalDamage specialMoveReady\n";

    this_thread::sleep_for(chrono::seconds(2));

    // a bit theme :>
    string url = "https://youtu.be/JgNTxDhGvsI?si=0el1VleeEyxC1xj7&t=131s";
    #ifdef _WIN32
        HWND console = GetConsoleWindow();
        system(("start " + url).c_str());
        SetForegroundWindow(console);
    #elif __APPLE__
        system(("open " + url).c_str());
    #else // Linux
        system(("xdg-open " + url + " >/dev/null 2>&1 &").c_str());
    #endif

    int slayerLevel, hp, hasTalisman, specialMoveReady, allyCount;
    int demonPresence, demonRank, bossHP, totalDamage;
    double breathingMastery, swordSharpness;
    char timeOfDay;

    cin >> slayerLevel >> hp >> breathingMastery >> hasTalisman
        >> timeOfDay >> demonPresence >> demonRank >> swordSharpness
        >> allyCount >> bossHP >> totalDamage >> specialMoveReady;
```

```

    // Scene 1: Power
    double power = slayerLevel*10 + hp/10.0 + breathingMastery*50.0;
    string rank = (power >= 120) ? "Hashira" : (power >= 80) ? "Elite" : "Novice";

    cout << "[Scene 1] Rank: " << rank << " (power = " << power << ")\n";

    // Scene 2: Gate
    cout << "[Scene 2] ";

    if (hasTalisman == 0) cout << "Denied: No talisman.\n";
    else if (timeOfDay != 'D' && timeOfDay != 'N') cout << "Warning: invalid timeOfDay.\n";
    else {
        bool night = (timeOfDay == 'N');
        string openGate = (night && demonPresence == 1) ?
            "Open silently.\n" : "Open cautiously.\n";
        cout << openGate;
    }

    // Scene 3: Strategy
    double adv = (101 + -demonRank*15) + swordSharpness*0.4 + allyCount*5;
    string strategy;

    if (!(adv < 100)) strategy = "Engage head-on";
    else if (adv >= 60) strategy = "Harass and probe";
    else strategy = "Retreat and regroup";

    cout << "[Scene 3] " << strategy << " (adv = " << adv << ")\n";

    // Scene 4: Final
    int finalHP = bossHP + -totalDamage;
    cout << "[Scene 4] ";

    if (finalHP <= 0) cout << "Boss defeated! (finalHP = 0)\n";
    else if (specialMoveReady == 1 && finalHP <= 50) {
        cout << "Use special move to finish! (finalHP = " << finalHP << ")\n";
    }
    else cout << "Withdraw to heal.\n";

    return 0;
}

```

4.1. Header

```
#include <iostream>
#include <iomanip>
#include <string>
#include <thread>
#include <chrono>
#ifdef _WIN32
#include <windows.h>
#endif
```

These libraries provide all core functionalities:

- **`<iostream>`** and **`<iomanip>`**: manage console input/output and numeric formatting.
- **`<string>`**: supports URL manipulation as a string object.
- **`<thread>`** and **`<chrono>`**: enable timed delays for enhanced user interaction.
- Conditional inclusion of **`<windows.h>`** ensures compatibility with Windows API functions used for console focus control.

4.2. Introduction Output

```
    cout << "\nWelcome come to THE INFINITY CASTLE !!\n\n";
    cout << "To survive, please enter (๑•`ㅂ•´)و✧ \n";
    cout << "slayerLevel hp breathingMastery hasTalisman timeOfDay demonPresence ";
    cout << "demonRank swordSharpness allyCount bossHP totalDamage specialMoveReady\n";
```

The introduction section greets the player, sets the context, and clearly lists the twelve required input parameters. This structure improves usability and prevents input errors by guiding the user on expected data order and types.

4.3. Opening the YouTube Theme

```

    this_thread::sleep_for(chrono::seconds(2));

```

Implements a 2-second pause using the C++ **`<thread>`** library. This controlled delay enhances the user experience, simulating a loading sequence or suspenseful moment before the main program proceeds.

```
    // a bit theme :>
    string url = "https://youtu.be/JgNTxDhGvsI?si=0el1VleeEyxC1xj7&t=131s";
    #ifdef _WIN32
        HWND console = GetConsoleWindow();
        system(("start " + url).c_str());
        SetForegroundWindow(console);
    #elif __APPLE__
        system(("open " + url).c_str());
    #else // Linux
        system(("xdg-open " + url + " >/dev/null 2>&1 &").c_str());
    #endif
```

This segment launches a YouTube soundtrack to accompany the simulation:

- **`system()`** executes the OS-specific command to open the link.
- **`#ifdef`** preprocessor directives ensure the code adapts across Windows, macOS, and Linux.
- The video starts at 131 seconds, aligning thematically with the intended moment in the song.
- On Windows, **`GetConsoleWindow()`** and **`SetForegroundWindow()`** are used to regain console focus after the browser opens.

4.4. Variable Declaration and Data Input

```
    int slayerLevel, hp, hasTalisman, specialMoveReady, allyCount;
    int demonPresence, demonRank, bossHP, totalDamage;
    double breathingMastery, swordSharpness;
    char timeOfDay;

    cin >> slayerLevel >> hp >> breathingMastery >> hasTalisman
        >> timeOfDay >> demonPresence >> demonRank >> swordSharpness
        >> allyCount >> bossHP >> totalDamage >> specialMoveReady;
```

The program uses mixed data types for efficient memory use:

- **`int`** for discrete attributes (levels, ranks, boolean flags).
- **`double`** for continuous values (mastery, sharpness).
- **`char`** for single-letter time representation (‘**`D`**’ or ‘**`N`**’).

Inputs are gathered in one statement for compactness, following the exact order displayed in the instructions.

4.5. Scene 1: Power Classification

Compute the total power index of the demon slayer using the formula:

$$power = slayerLevel \times 10 + \frac{hp}{10} + breathingMastery \times 50$$

Classification rules:

- $power \geq 120 \Rightarrow$ Hashira
- $80 \leq power < 120 \Rightarrow$ Elite
- $power < 80 \Rightarrow$ Novice

Output format:

`[Scene 1] Rank: <Hashira/Elite/Novice> (power = <value>)`

```
// Scene 1: Power
double power = slayerLevel*10 + hp/10.0 + breathingMastery*50.0;
string rank = (power >= 120) ? "Hashira" : (power >= 80) ? "Elite" : "Novice";

cout << "[Scene 1] Rank: " << rank << " (power = " << power << ")\n";

```

This block calculates a player's power index using weighted parameters.

A ternary operator efficiently classifies ranks:

- $\geq 120 \rightarrow$ Hashira
- $\geq 80 \rightarrow$ Elite
- $< 80 \rightarrow$ Novice

4.6. Scene 2: Infinity Castle Gate

Check whether the gate can be opened based on the following rules:

- If `hasTalisman == 0` $\Rightarrow$ `Denied: No talisman.`
- If `timeOfDay` is not '`D`' or '`N`' $\Rightarrow$ `Warning: invalid timeOfDay.`
- If `timeOfDay == 'N'` and `demonPresence == 1` $\Rightarrow$ `Open silently.`
- Otherwise $\Rightarrow$ `Open cautiously.`

Output format:

`[Scene 2] <result>`

```
    // Scene 2: Gate
    cout << "[Scene 2] ";

    if (hasTalisman == 0) cout << "Denied: No talisman.\n";
    else if (timeOfDay != 'D' && timeOfDay != 'N') cout << "Warning: invalid timeOfDay.\n";
    else {
        bool night = (timeOfDay == 'N');
        string openGate = (night && demonPresence == 1) ?
            "Open silently.\n" : "Open cautiously.\n";
        cout << openGate;
    }

```

Scene 2 uses multi-layered conditional checks:
The first condition blocks access if the player lacks a talisman.
The second validates correct input for time of day.
If valid, it defines a Boolean **`night`** and uses a ternary operator to determine the correct message:

- At night and demon present → **`"Open silently."`**
- Otherwise → **`"Open cautiously."`**
- This structure highlights the student's proficiency with nested logic and logical operators (**`&&`**, **`!=`**).

4.7. Scene 3: Battle Strategy

Compute the battle advantage index:

$$adv = (101 - demonRank \times 15) + swordSharpness \times 0.4 + allyCount \times 5$$

Classify the strategy:

- $adv \geq 100 \Rightarrow$ `Engage head-on`
- $60 \leq adv < 100 \Rightarrow$ `Harass and probe`
- $adv < 60 \Rightarrow$ `Retreat and regroup`

Output format:

`[Scene 3] <strategy> (adv = <value>)`

```
    // Scene 3: Strategy
    double adv = (101 + -demonRank*15) + swordSharpness*0.4 + allyCount*5;
    string strategy;

    if (!(adv < 100)) strategy = "Engage head-on";
    else if (adv >= 60) strategy = "Harass and probe";
    else strategy = "Retreat and regroup";

    cout << "[Scene 3] " << strategy << " (adv = " << adv << ")\n";
```

This section calculates an advantage score that governs the recommended battle strategy.

The formula balances demon strength (negative weight) and ally support (positive reinforcement). The use of a negation operator **`!(adv < 100)`** instead of **`(adv >= 100)`** adds syntactic uniqueness while preserving logic.
Three strategic tiers are defined:

- High advantage → **`Engage head-on`**
- Moderate advantage → **`Harass and probe`**
- Low advantage → **`Retreat and regroup`**

4.8. Scene 4: Final Outcome

Compute the boss's remaining HP:

$$finalHP = bossHP - totalDamage$$

Apply the following logic:

- If $finalHP \leq 0 \Rightarrow$ `Boss defeated! (finalHP = 0)`
- If $finalHP > 0$, `specialMoveReady == 1` and $finalHP \leq 50 \Rightarrow$ `Use special move to finish!`
- Otherwise $\Rightarrow$ `Withdraw to heal.`

Output format:

`[Scene 4] <result>`

```
// Scene 4: Final
int finalHP = bossHP + -totalDamage;
cout << "[Scene 4] ";

if (finalHP <= 0) cout << "Boss defeated! (finalHP = 0)\n";
else if (specialMoveReady == 1 && finalHP <= 50) {
    cout << "Use special move to finish! (finalHP = " << finalHP << ")\n";
}
else cout << "Withdraw to heal.\n";
```

The boss's final health is calculated by simple subtraction written in an alternate arithmetic form (**`+ -totalDamage`**) instead of (**`- totalDamage`**), adding stylistic distinctiveness.

The condition structure ensures three possible outcomes:

- Boss defeated → victory message **"`Boss defeated! (finalHP = 0)`"**
- Near victory with special move ready → encouragement to finish **"`Use special move to finish!`"**
- Otherwise → retreat recommendation **"`Withdraw to heal`"**

5. Results

5.1. Example 1

Input

```
8 950 0.8 1 N 1 3 72.5 2 420 380 1
```

Output

```
[Scene 1] Rank: Hashira (power = 215.0)
[Scene 2] Open silently.
[Scene 3] Harass and probe (adv = 95.0)
[Scene 4] Use special move to finish! (finalHP = 40)
```

Discussion

Each computed output matches the logical expectation of the algorithm:

- The rank **"Hashira"** corresponds to a high computed power index (215.0 ≥ 120).
- **"Open silently"** confirms that the talisman is present, **timeOfDay = 'N'**, and **demonPresence = 1**.
- The **"Harass and probe"** strategy results from a mid-tier advantage score (95.0 between 60-100).
- In Scene 4, the boss still has 40 HP left, but the special move is ready → correct message: **Use special move to finish!**

This verifies that all arithmetic and conditional operations are performing correctly.

5.2. Example 2

Input

```
4 450 0.3 0 D 0 5 60.0 0 500 200 0
```

Output

```
[Scene 1] Rank: Elite (power = 100.0)
[Scene 2] Denied: No talisman.
[Scene 3] Retreat and regroup (adv = 50.0)
[Scene 4] Withdraw to heal.
```

Discussion

- Scene 1: The total power = **4*10 + 450/10 + 0.3*50 = 40 + 45 + 15 = 100.0**, which classifies the player as **Elite**.

- Scene 2: Since **`hasTalisman == 0`**, access is denied immediately - no further evaluation is needed.
- Scene 3: With demonRank = 5, allyCount = 0, and moderate swordSharpness, the computed advantage (**`50.0`**) leads to **`"Retreat and regroup"`**.
- Scene 4: The boss still has positive HP remaining and the special move isn't ready → correct outcome **`Withdraw to heal`**.

This input verifies the fail conditions (no talisman, low stats) and ensures that early-exit conditions are handled properly.

5.3. Example 3

Input

```
10 1200 1.0 1 D 0 2 85.0 3 300 350 0
```

Output

```
[Scene 1] Rank: Hashira (power = 270.0)
[Scene 2] Open cautiously.
[Scene 3] Engage head-on (adv = 120.0)
[Scene 4] Boss defeated! (finalHP = 0)
```

Discussion

- Scene 1: A very high power score (270.0) correctly results in Hashira rank.
- Scene 2: It is daytime (**`timeOfDay = 'D'`**), so despite demon presence being false, the program correctly selects **`"Open cautiously."`**
- Scene 3: The advantage value (**`120.0`**) is above 100 → Engage head-on is chosen. Formula check: **`adv = 101 - 2*15 + 85*0.4 + 3*5 = 101 - 30 + 34 + 15 = 120`**.
- Scene 4: The boss's final HP = 300 - 350 = -50, which is ≤ 0 → **`Boss defeated! (finalHP = 0)`** is the correct message.

This test confirms proper functioning of the highest-rank branch, ensuring all upper thresholds produce correct outcomes.

5.4. Example 4

Input

```
6 800 0.5 1 X 1 4 70.0 1 500 200 0
```

Output

```
[Scene 1] Rank: Hashira (power = 165.0)
[Scene 2] Warning: invalid timeOfDay.
[Scene 3] Harass and probe (adv = 74.0)
[Scene 4] Withdraw to heal.
```

Discussion

- Scene 1: Power = `6*10 + 800/10 + 0.5*50 = 60 + 80 + 25 = 165.0`, within the **"Hashira"** range.
- Scene 2: Because `timeOfDay = 'X'`, the program properly triggers the **"Warning: invalid timeOfDay."** branch.
- Scene 3: Advantage = `(101 - 4*15 + 70*0.4 + 1*5) = 101 - 60 + 28 + 5 = 74.0` → `Harass and probe.`
- Scene 4: `BossHP = 500, totalDamage = 200` → remaining HP = 300. No special move ready → `Withdraw to heal`.

This test verifies error handling for invalid input (`timeOfDay`) and ensures unaffected continuation in later scenes.

5.5. Example 5

Input

```
7 700 0.6 1 N 0 6 50.0 5 800 750 0
```

Output

```
[Scene 1] Rank: Hashira (power = 170.0)
[Scene 2] Open silently.
[Scene 3] Retreat and regroup (adv = 56.0)
[Scene 4] Withdraw to heal.
```

Discussion

- Scene 1: Power = `7*10 + 700/10 + 0.6*50 = 70 + 70 + 30 = 170.0` → `Hashira`
- Scene 2: It is nighttime with talisman present; even though `demonPresence = 0`, the logical fallback still allows entry silently.
- Scene 3: A high demon rank (6) severely reduces advantage: `adv = 101 - 6*15 + 50*0.4 + 5*5 = 101 - 90 + 20 + 25 = 56.0` → `Retreat and regroup`.
- Scene 4: Final `HP = 800 - 750 = 50`, but the special move isn't ready → correct message: `Withdraw to heal`.

This scenario tests borderline conditions, verifying the lower edge of advantage thresholds and the absence of the special move trigger.

6. Conclusion

6.1. Finale

Silence falls.
The echo of steel fades into the trembling heartbeat of the Infinity Castle. The dust settles - crimson embers swirling through the air like fireflies in a dying dream.

The Slayer stands still, his breathing steady yet strained. Every decision - every calculation - every breath of mastery - has led to this moment.

His power was tested.
His courage measured.
And the Castle… it watched, shifting and whispering, weighing the worth of his soul.

If victory is his - the gates shatter open, and the first light of dawn spills across the shattered tiles.
The Boss Demon dissolves into ash, its cry fading into silence, and the Slayer's blade - gleaming faintly - rests upon his shoulder. The path is clear. The curse is lifted.

But if defeat has claimed him -
the walls of the Castle close once more. The corridors twist upon themselves, sealing his fate in endless echoes of regret. His spirit becomes one with the Castle - another whisper in its infinite halls.

Either way…
the Infinity Castle remembers.

It stands eternal, waiting for the next warrior brave enough to challenge its depths.
Because within this labyrinth of fate, power alone is not enough - only the harmony of strength, will, and judgment can determine who survives the night.

And as the screen fades to black, the last words burn in silence:

"In the Infinity Castle, every choice is destiny… and destiny is earned."

6.2. Epiloque

The Infinity Castle program successfully integrates creative narrative design with formal computational logic:

- Strong understanding of conditional structures,
- Competent data type selection and arithmetic modeling,
- Effective cross-platform system handling,
- Readable, efficient, and logically consistent C++ code.

The project fulfills objectives, and the deliberate thematic integration and adherence to structured programming make this a complete and well-executed academic submission.

7. AI-used Declaration

This report was prepared with assistance from **ChatGPT** for the purposes of:

- academic language refinement, structural formatting, and grammar improvement,
- generating and studying the programming code for **Opening the YouTube Theme**,
- further fascinate the battle plot, suitable to the moving and action theme.

All programming code, testing, and analysis were performed independently by the student.
The author takes full responsibility for the correctness and originality of both code and content.